Semesterproject: Intelligent Software Systems

FINAL REPORT

Group 3

Gabriele Solazzo

Aleksandra Kwiatkowska

Gabija Staskeviciute

Luigi Colluto

Manish Raj Moriche

Mats Haertel

Teacher and supervisor

Devender Kumar

Riccardo Terrenzi

TEK Faculty - Sønderborg Campus

Software Engineering, Semester 4 - 2026

Course ID: T630012401

Abstract

Table of contents

(just to point out what we will have (based on last semester)

1. Introduction

This project was developed as part of the semester curriculum in Software Engineering and concerns the design and implementation of a web-based platform for supporting medical consultations through recording, transcription, and document generation. The system is intended to support doctors, patients, and administrators by providing a secure and user-friendly application for managing consultation-related information and follow-up communication.

The project addresses practical challenges within medical consultation workflows, where documentation is often performed manually while the consultation is taking place. Doctors need efficient tools for capturing consultation content, preparing documentation, and managing follow-up material, while patients need a reliable way of receiving relevant information after the consultation. To support these needs, the application allows the doctor to record an ongoing consultation, process the spoken content, and generate a draft PDF containing consultation-related information and suggestions.

Rather than implementing a standalone appointment scheduling service, the project focuses specifically on support during and after an ongoing consultation. In the intended context, healthcare providers are assumed to already use existing scheduling solutions, and because the application is designed according to component-based principles, such functionality can be integrated as an external component rather than implemented directly within the system.

From a technical perspective, the project reflects the semester's focus on Artificial Intelligence, Component-based Systems, Algorithms and Data Structures, and Designing Software Systems with Embedded Elements. This is demonstrated through AI-assisted transcription and summarization, a modular component-based architecture, structured handling of consultation data, and secure processing of sensitive medical information. Particular emphasis is placed on usability, privacy, and efficient communication between the involved users. In addition, the project builds on the original proposal for OPD-Vertex, which emphasized local processing, privacy-aware handling of medical data, and loosely coupled system design as central goals. The resulting system demonstrates how software can support documentation and communication in medical consultation settings.

1.1. Motivation

Medical consultations often require doctors to divide their attention between interacting with patients and documenting notes, prescriptions, and other relevant information. This can drastically reduce consultation efficiency, increase the risk of missing important details, and delay the preparation of post-consultation documentation. At the same time, the growing use of digital and AI-assisted healthcare tools raises important concerns regarding privacy, legal compliance, protection of sensitive patient data, and dependence on external cloud services.

This project is motivated by the need for a more efficient and privacy-conscious documentation workflow during medical consultations. By supporting consultation recording, transcription, draft generation, and follow-up communication in a single system, the application aims to reduce

administrative overhead while improving the quality and accessibility of consultation-related documentation. The project is further motivated by the opportunity to explore how AI-assisted tools can support doctors without removing clinical responsibility from the final decision-making process.

1.2. Objective

The objective of this project is to design and implement a secure web-based consultation support system that assists doctors during and after medical consultations. The system aims to record consultation audio, convert spoken dialogue into text, and generate a draft PDF containing suggestions and consultation-related information that the doctor can review, edit, approve, or decline before further processing.

Once approved by the doctor, the system supports continued handling of the consultation outcome by allowing relevant information, such as prescription details and follow-up material, to be sent to the patient by email. In addition, the system provides functionality for viewing patient details, accessing doctor information, downloading generated PDF documents, and presenting relevant information through a dashboard.

1.3. Delimitation

The application is not intended to provide a complete medical administration system and does not include broader services such as full clinical record management, or full integration with external healthcare infrastructure. The project focuses on supporting activities during and after an ongoing consultation. In the intended deployment context, healthcare providers are assumed to already rely on established scheduling systems. Therefore, in line with the systems's component-based architecture, scheduling is treated as an external concern that can be integrated through well-defined interfaces.

Likewise, the generated transcription, summaries, suggestions, and PDF drafts are intended solely as decision-support material for the doctor. They are not autonomous medical outputs, and final responsibility for reviewing, editing, approving, and declining the generated content remains with the doctor before any information is processed further or shared with the patient.

The project also does not include long-term clinical testing, user studies with external patients or healthcare providers, or deployment in a live medical environment. Advanced compliance certification, large-scale deployment, and full integration with existing healthcare systems are therefore considered outside the scope of this project.

2. Methodology - add pictures of the sprints from Trello

2.1 Teamwork and Collaboration

Tasks were distributed in pairs throughout most of the project, which helped with collaboration and peer review. The workflow was organized in Trello across 5 sprints, allowing the team to track and manage tasks. In Sprint 1, we focused on planning the project structure by preparing the project proposal and creating different diagrams, and we also designed the database. In Sprint 2, we set up the application environment, designed the first draft of the UI, configured both SQL and NoSQL databases and implemented authentication features for patients, doctors and admins. In Sprint 3, we implemented audio capture and speech-to-text transcription, fixed and improved the database and prepared the project presentation for the midterm seminar. In Sprint 4, we implemented documented versioning and editing, added email sending functionality, integrated LLM capabilities for transcription, suggestion and reports, and completed the user interface implementation. In Sprint 5, we fixed the last bugs, updated the diagrams, created the presentation for the last seminar and finished the report. GitHub was used to share and manage the code, allowing the team to collaborate efficiently

while keeping all contributions organized in one place. The team met around twice per week to review progress, coordinate tasks and resolve any blockers. For communication, we used mainly WhatsApp for daily updates and Discord for sharing files, data and other project resources.

2.2 Application of Course Knowledge

The application was developed using a component-based architecture, following the concepts learned during the Component-Based Systems course. This approach made it easier for the team members to work in parallel on different parts of the system without interfering with each other's progress. By separating the application into independent components, development became more organized, scalable and easier to maintain over time.

Testing was carried out throughout the development process, applying principles from the Designing Software Systems with Embedded Elements course. Instead of leaving testing only for the final stages, features were regularly checked and improved as they were implemented, helping the team identify issues earlier and maintain a more stable system overall.

An LLM was integrated in the application to support many features (e.g. Suggestive Mode). This part of the project combined ideas from both the Artificial Intelligence course and the Algorithms and Data Structure course, especially in terms of data processing, optimization and AI-driven functionalities.

The team also applied knowledge gained during the previous semester by using Docker for containerization, which simplified deployment and ensured a more consistent development environment across all machines, a concept learned from the Operating Systems and Distributed Systems course. In addition, concepts from the Data Management course were used to design the database architecture, which was split into two systems: MySQL for structured relational data and MongoDB for document-oriented storage. This separation allowed each database to be used where it was more appropriate, improving both efficiency and overall data organization in the application.

3. Problem Analysis

3.1 Risks and Limitations of Manual Documentation

A large-scale descriptive study[1] analysing approximately 100 million patient encounters with about 155 thousand physicians found that average time spent on Electronic Health Records (EHR) is about 16 minutes per encounter. When contrasted with a typical 15 to 20 minute patient consultation, this data indicates that administrative tasks take almost as much time as the medical evaluation itself.

Furthermore, the American Medical Association highlights[2] that administrative tasks occupy up to 5.8 hours of a standard 8-hour shift, meaning doctors are forced to multi-task by actively listening and typing summaries. This divided focus increases the risk of documentation errors or missing critical diagnostic verbal cues.

To eliminate these bottlenecks the system must capture information passively (via audio) rather than relying on the manual input. Moreover, this automation must function as a decision-support tool, generating editable drafts and real-time suggestions in order to ensure that the doctor holds final clinical responsibility and oversight.

Sources (put as references later):

https://pubmed.ncbi.nlm.nih.gov/31931523/

https://www.tmlt.org/resource/using-ai-medical-scribes-risk-management-considerations#:~:text=The%20AMA%20found%20that%20physicians,3)

3.2 Security and Privacy in AI-Assisted Healthcare

According to Article 9 of the General Data Protection Regulation (GDPR), medical data such as clinical notes, transcriptions and prescriptions is classified as a "Special Category of Personal Data". This classification significantly restricts how it can be collected, stored and processed.

Existing AI documentation tools rely on transmitting data to external cloud servers which directly conflicts with the GDPR principles regarding data sovereignty and purpose limitation. Furthermore, this cloud dependency introduces a risk of the system downtime in case the external network fails.

In order to prevent both the risks of data leaks and vulnerability of internet dependency, the application must operate as a locally hosted system using open-source LLMs deployed on the clinic's internal network. This architecture aims that all audio processing, transcription, and document generation happen locally, ensuring that sensitive data remains fully protected and private.

3.3 Architectural Challenges

Traditional EHR systems are typically built as monolithic or tightly coupled structures. This architecture creates a major issue because updating a single feature requires modifying and risking stability of the entire system. This becomes a critical bottleneck when dealing with Artificial Intelligence and speech-to-text components, which evolve rapidly and require flexible setup to be updated independently.

In addition, operating on such diverse datasets as live audio metadata, raw speech-to-text transcriptions and generated PDFs creates a direct architectural conflict. While user authentication and patient records require strict relational consistency, generated transcripts and audio files require high schema flexibility.

Forcing these conflicting data types into a single database model leads to severe performance bottlenecks. Therefore, the system must manage the need for strict relational consistency for user administration and dynamic schema flexibility for AI data processing.

3.4 Problem Statement

Based on the analysis above, the core challenge of this project is that modern medical documentation tools place a heavy administrative burden on doctors, yet existing cloud-based AI automation tools introduce unacceptable data privacy risks and operational vulnerabilities. Furthermore, traditional monolithic architectures and strict database structures cannot handle rapid evolution of local AI tools and the combination of structured and unstructured clinical datasets. Therefore this project seeks to answer the following question: How can a software system be designed and implemented to automate medical consultation documentation using local AI assistance, while ensuring data privacy, security and architectural flexibility?

4. Requirements

4.1 Must Have Requirements

Functional Requirements:

- **F-1:** The system must manage a doctor consultation queue.
- **F-2:** The system must capture consultation audio locally.
- **F-3:** The system must transcribe speech to text locally.
- **F-4:** The system must generate structured clinical notes.
- **F-5:** The system must generate prescription drafts.
- **F-6:** The system must allow doctors to edit and approve clinical notes and prescriptions.
- **F-7:** The system must generate prescription PDFs.
- **F-8:** The system must prescriptions securely via email.

Non-Functional Requirements:

- **NF-1 (Privacy):** All AI processing must be performed locally.
- **NF-2 (Security):** The system must use authentication and role-based access.
- **NF-3 (Encryption)**: The system must ensure encryption for all sensitive data.
- **NF-4 (Modularity):** Components must be loosely coupled and replaceable.
- **NF-5 (Reliability):** Consultation data must be stored safely.
- **NF-6 (Auditability):** The system must log important actions and edits.

4.2 Should Have Requirements

Functional Requirements

- **F-9:** The system should display a live transcript during consultation.
- **F-10:** The system should allow doctors to view past consultations.

Non-Functional Requirements

- **NF-7:** (Usability): The interface should be user-friendly and efficient.
- **NF-8:** (Performance): On the target machine, local transcription agent should operate with RTF $\leq$ 1.0 (real-time transcription). For a typical consultation transcript, the system should generate structured notes + prescription draft + clinical alerts within $\leq$ 40 seconds on recommended hardware (GPU $\geq$ 12GB VRAM, 32GB RAM) and within $\leq$ 120 seconds on minimum hardware (CPU-only, 16GB RAM).

4.3 Could Have Requirements

Functional Requirements

- **F-11**: Support for multiple doctors for sharing documents (e.g. a sharable report that could be sent to a different doctor than the family doctor).

Non-Functional Requirements

- **NF-9:** The system could allow administrators to configure for how long consultation data, transcripts, and prescriptions are stored (TBD) and must support secure deletion of expired records.

4.4 Will Not Have Requirements

Non-Functional Requirement

- **NF-10:** The system will not use cloud-based AI services.
- **NF-11:** The system alone (excluding the doctor) is not enough for diagnosing the user.

5. Design
6. Implementation
7. Validation
8. Conclusion

5. Design

In this chapter we describe the design of OPD-Vertex, focusing on the principles that we followed to structure the system, the main components and their interactions, the data model and the cross-cutting concerns of our application.

5.1 Design Goals and Principles

We designed OPD-Vertex as a privacy-first, locally hosted clinical assistant for outpatient departments. The architecture is based on five principles that we used to guide every design decision during development: local-first execution (all the sensitive processes run inside the clinic environment, so no consultation data is sent to a third-party cloud LLM), component-based design (the system is split into clearly bounded modules, each one with its own application service and domain contracts), loose coupling through ports and adapters (each external dependency is exposed as a domain port and implemented by an adapter, so the application logic never depends on a concrete library), replaceability of AI components (transcription, LLM and suggestive-review are independent ports, so we could swap them without changing the application), and human-in-the-loop approval (no AI output is ever written to the relational database without doctor approval). Thanks to these principles, the system stays maintainable through a layered architecture and is easy to extend, because each component can be swapped without rewriting the core.

5.2 System Overview and Architecture

Before writing the code, we sketched the main user journeys and the screen layouts in Figma. We kept the mockups intentionally lightweight (since UI was not the focus) and we used them mainly to agree on a few things: the doctor login, the doctor dashboard, the consultation page with audio recording, the draft review page and the patient view. The three user roles that we implemented today (doctor, admin and patient) were already defined in these first mockups. The mockups also evolved during development, in particular the review page, that was split into view, editor and print modes once we decided to support markdown editing.

The target architecture of OPD-Vertex is a set of cooperating services organized around the clinical workflow: identity, patient, consultation, transcription, LLM orchestration, suggestive review, PDF, notification and audit. Public endpoints sit behind an API gateway and AI components run on dedicated GPU-enabled nodes. Each service owns its own datastore, in order to preserve the loose coupling that we set as a design goal. In the current system we realized this design on a smaller scale: the logical components live as Python modules inside a single FastAPI application, the transcription module runs as a separate Whisper sidecar, the LLM runs in Ollama, MySQL holds the relational data and MongoDB holds the AI documents. All the services are packaged as Docker containers and orchestrated together with Docker Compose, so we have a reproducible setup across all our machines and a clear migration path towards the target architecture. Since all the inference happens locally, no consultation data ever leaves the host.

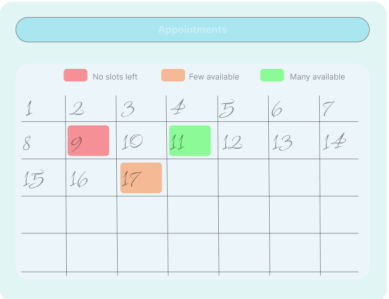


Figure 5.1 - Initial Figma prototype of the doctor dashboard and consultation screens.

5.3 Component-Based System Design

We decided to decompose the system into a set of loosely coupled components, where each one has a single responsibility and can be developed, tested or replaced independently from the others. Figure 5.2 shows the logical component diagram that we used during the design phase. In the current system many of these components live as modules inside the Local Backend App, while the diagram represents the logical boundaries that we want to support.

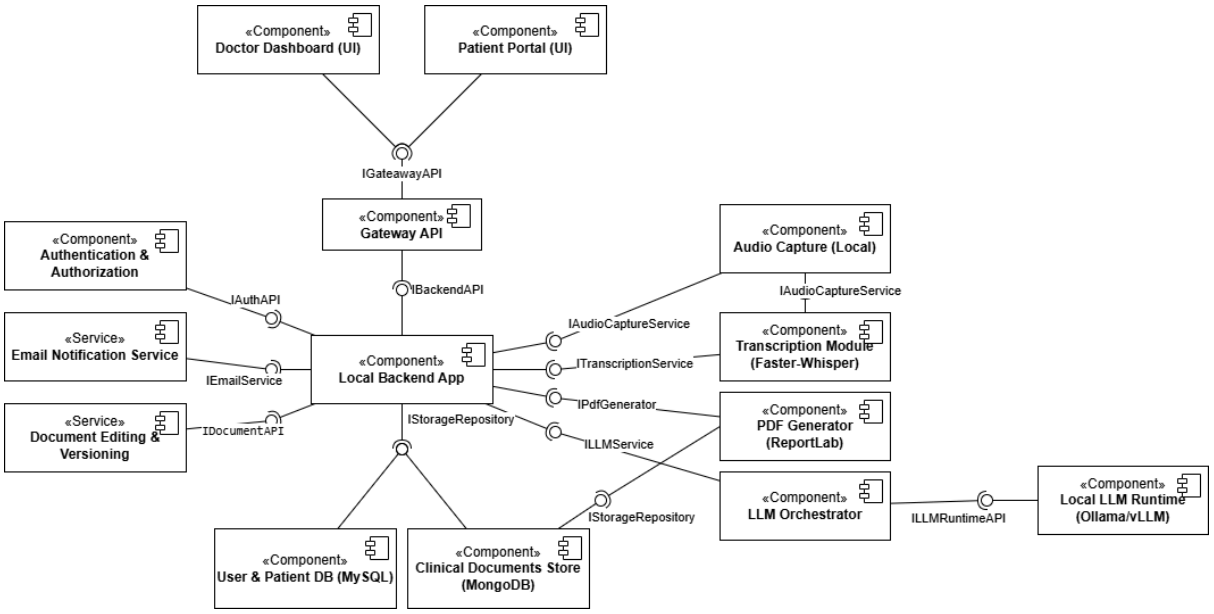


Figure 5.2 - Component diagram showing the main components of OPD-Vertex and their interfaces.

On the user-facing side, the Doctor Dashboard (UI) is the main interface used by doctors and gives access to the patient list, the consultation recording, the draft review and the prescription approval, while the Patient Portal (UI) is used by patients to log in and read their own consultations and prescriptions. Both UIs are server-rendered with Jinja2 templates and they communicate with the backend only through the Gateway API (IGatewayAPI), that is in charge of request routing, validation and authorization before passing the request to the Local Backend App (IBackendAPI), the orchestrator of the whole workflow that owns the application services and depends only on the abstract interfaces exposed by the other components.

Three components handle the cross-cutting concerns: Authentication & Authorization (IAuthAPI) validates credentials, issues JWT tokens and enforces role-based access control; the Email Notification Service (IEmailService) sends the patient summaries through SMTP; the Document Editing & Versioning service (IDocumentAPI) manages the lifecycle of the generated reports, including markdown edits and version history, so that every revision made by the doctor can be traced back.

The AI pipeline starts from Audio Capture (Local) (IAudioCaptureService), that captures the consultation audio on the client side and streams it to the Transcription Module (Faster-Whisper, ITranscriptionService), where speech is converted into text locally. The LLM Orchestrator (ILLMService) manages prompts, retries and structured output validation, while the actual generation is delegated to the Local LLM Runtime (Ollama/vLLM, ILLMRuntimeAPI), and the PDF Generator (ReportLab, IPdfGenerator) renders the approved markdown report into a printable document. Finally,

for persistence we use two databases accessed through a common IStorageRepository interface: the User & Patient DB (MySQL) stores the structured relational data, and the Clinical Documents Store (MongoDB) stores the semi-structured AI artefacts. The reason behind this dual-database choice is discussed in Section 5.5.

5.4 Aspect-Oriented Programming for Cross-Cutting Concerns

Some concerns like logging, performance measurement, error capture and request tracing touch almost every layer of the system, but they do not really belong to the business logic of any single module. If we had to write a logger.info(...) call in every service method, the code would become full of boilerplate and the business logic would be tightly coupled with the observability stack. For this reason, we decided to treat these concerns as cross-cutting concerns and to handle them with Aspect-Oriented Programming (AOP).

Our AOP layer is based on a class-level decorator that automatically wraps every public method of a class with entry, exit and error logging, and measures the wall-clock duration. We also added a middleware that logs every HTTP request (method, path, client IP, status code, duration) without changing the route handlers. Thanks to this approach, the application services and the repository classes stay free of log statements and at the same time produce uniform entries with consistent duration units, that makes log filtering much easier. A future migration to a tracing or metrics backend would also require to change just one file instead of refactoring the entire codebase.

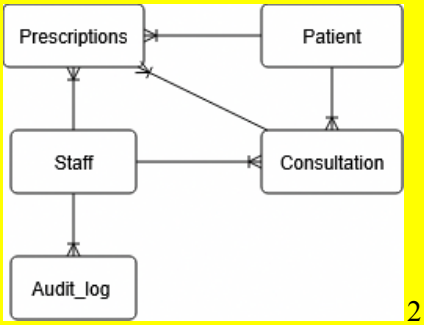
Picture needed?

5.5 Data Design and Dual-Database Architecture

We decided to use a dual-database design in order to separate the system of records from the iterative AI artefacts. The relational store gives us consistency for the clinically meaningful data, while the document store gives us the flexibility that we need for evolving AI outputs.

MySQL 8.4 - system of record. We use five tables to hold the structured data: staff (doctors and admins with role, hashed password, license and specialization), patients (demographics, allergies, medical history, hashed password), consultations (the link between a doctor and a patient, with a status enum: recording → transcribing → processing → review → approved/rejected/cancelled), prescriptions (finalised and versioned prescription records, with a unique constraint on consultation + version, that we use to support re-approval as new versions), and audit_logs (an append-only log of the significant actions in the system). (Maybe a summary of this and then a diagram?)

MongoDB 7 - document store. On the MongoDB side, we use different collections to hold the semi-structured artefacts of the AI pipeline: consultation_documents for transcripts and working notes, generated_documents for the drafted reports and the suggestive reviews, prescription_artifacts for the generated PDFs, llm_prompts for the versioned prompt templates, and email_templates for the editable patient communications. The consultation_id is the shared linkage key between the two databases. The iterative and fast-changing AI artefacts live in MongoDB, while the legally meaningful record (the approved prescription) is moved into MySQL only after the doctor approves the draft. Thanks to this separation, we also enforce the human-in-the-loop principle at the data layer, since the relational record is always the result of a clinician decision.



2 small one (add mongoDB in line)

Figure 5.3 - Database diagram showing the relational schema in MySQL and the document collections in MongoDB.

5.6 Main Workflow and AI Pipeline

The end-to-end consultation flow is implemented as three smaller workflows that compose into a single user journey: consultation creation, AI processing and prescription approval. The doctor opens the patient list, selects a patient and starts a consultation, the system creates a new record with status recording and the audio is captured in the browser and streamed to the backend through a WebSocket, that forwards the chunks to the Whisper sidecar in near real time. We also store partial transcripts, so that the session can survive a disconnect. When the doctor stops the recording, the consultation moves to processing and the AI pipeline starts.

The AI pipeline itself is made of two LLM calls in series, followed by a safety pass. The transcript normalizer converts the raw transcript into a structured JSON object that filters out the speech artefacts, then the clinical-note generator combines the normalized transcript with the patient context (age, allergies, medical history) and asks Qwen3:8b to produce a structured object containing the diagnosis, medications, instructions, follow-up date, lab tests, imaging and a markdown rendering of the full report. Finally, the suggestive mode performs a second pass: an LLM-based clinical safety review is merged with deterministic rule-based checks (missing dose, frequency or duration, allergy and contraindication conflicts) and the combined output is shown to the doctor as advisory alerts. The suggestions are explicitly a support tool, so nothing is written to the prescription unless the doctor accepts it during the editing phase.

The doctor then opens the review page, optionally edits the markdown report and either approves or rejects the draft. If the draft is approved, the system builds a Prescription from it, saves it in MySQL with is_approved = true and the next version number, marks the generated document as APPROVED and sets the consultation status accordingly. Once approved, the doctor can also download the PDF and send a plain-text summary email to the patient (the email service raises an error if these actions are attempted before approval). Both the prompts used by the pipeline are stored as versioned records in MongoDB, so administrators can edit them at runtime without having to redeploy the application.

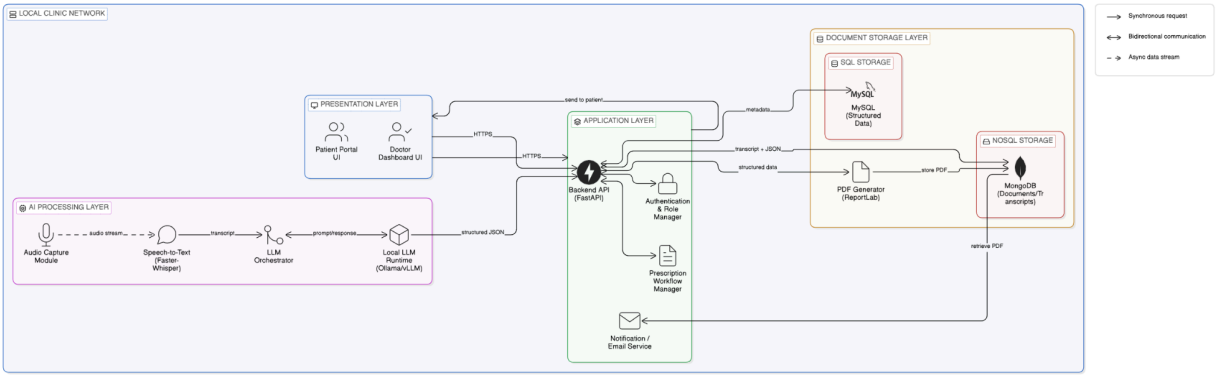


Figure 5.4 - End-to-end consultation sequence diagram (audio capture → transcription → LLM → review → approval).

5.7 Privacy, Security and Error Handling

In our design we treat the clinical data as data that must never leave the local environment, and the security model is built around this constraint. Both speech-to-text and the LLM run as local services, so the architecture has no path that could send transcripts or reports to a remote provider. The login is handled with bcrypt password hashing and short-lived JWT tokens stored in an HTTP-only cookie, role-based access control protects every route that mutates clinical content, and patients can see only their own consultations and prescriptions. The audit log records the significant actions (user, role, action, target, IP, timestamp and JSON details), including the administrative changes to prompts and templates. Two more design choices reinforce the privacy posture: the manual approval gate makes sure that no AI output is written into the relational prescriptions table without explicit doctor approval, and the email gating allows patient summaries to be sent only after approval.

For the error handling we decided to follow a small and predictable model. Domain-level problems raise a ValueError, that the API translates into 400 or 404 responses, while unexpected failures become 500. Transcription errors leave the consultation in its current state so the doctor can retry, LLM errors are caught by Pydantic validation and the drafts are left unchanged so the doctor can regenerate the report, while PDF and email failures return 500 without altering the underlying record. The doctor can always edit the draft, regenerate the report, regenerate the suggestive review or reject the draft entirely. We do not claim full regulatory compliance in this prototype. Hardening steps such as TLS termination, stronger authorization tests and at-rest encryption are treated as future deployment requirements.

5.8 UI Overview

We kept the UI intentionally simple and task-focused. Each screen serves one role and one job, and every page shares a common base layout and CSS bundle. The doctor dashboard is the role-aware home page for the clinicians, the consultation page hosts the live recording and the transcription, and the review page is the core human-in-the-loop screen, since it shows the AI draft, the structured fields, the suggestive-mode alerts, an editor mode for the markdown corrections and the approve, reject, print and email actions. The patient portal is read-only with respect to the clinical data, and the admin configuration view lets administrators edit prompts and email templates with an audit trail.

5.9 Scope Decisions and Future Enhancements

The project specification mentions a patient booking system among the features that the platform should support, but, after discussing it with the supervisor, we decided not to build an in-house scheduler and to expose a clean integration boundary instead. The reasons are mainly two: clinics usually already have a scheduling solution, often based on a national identity and e-health backbone like MitID in Denmark, so duplicating that functionality would not add much value; and the component-based architecture allows us to defer scheduling cleanly, because patients and consultations are independent modules with stable contracts, so an external scheduler can pre-create consultations through the same APIs that the rest of the system uses. As a result, doctors today create consultations directly from the patient list, and the full AI pipeline works end-to-end from there.

We also identified several extensions that would strengthen the platform for a real deployment. All of them fit within the existing component boundaries, that is in itself an indication that our architecture supports the substitutability we were aiming for:

- A queued retry layer for transient Whisper or Ollama failures, so that transient errors do not stop the current step anymore.
- Hardened security for the deployment phase: TLS termination, request-level authorization tests and at-rest encryption.
- An expanded set of suggestive-mode rules, including a broader drug-interaction database and stricter dosing checks.

- A booking component, that would expose a patient-facing calendar, slot management and reminders, plugged in alongside the existing modules.

6. Implementation

This chapter outlines the implementation approach for the OPD Vertex application. It will include the development process, core components, and technologies used. The system follows component-based architecture, with a backend build on FastAPI (Python). To store the variety of data, the system is using MySQL and MongoDB to create databases. Advanced features such as audio transcription and clinical summarization are handled by Faster-Whisper and Ollama/vLLM respectively. ReportLab is used for PDF generation, and emails are sent using a custom SMTP sender. The whole application is containerized using docker for consistent deployment over different environments. Configuration and Logging are managed with Pydantic.

6.1. API Layer & Routing

The API layer is built using FastAPI, to define and serve HTTP endpoints. This Layer acts as the main entry point for all interactions coming from the users.

Structure: The main API gateway is defined in router.py. This file creates the central APIrouter and includes sub-routers for each domain (e.g., auth, patients, consultations, etc.). Each of those sub-routers is responsible for a specific set of endpoints, leading to a clear separation of concerns and easy extensibility.

Dependency Injection: FastAPI's dependency injection is used to manage auth, database sessions, and other shared resources. The dependencies are defined in deps.py and injected into endpoint functions. This creates a consistent and secure system for shared resources.

Endpoint Definition: Each domain has its own router module. In each file endpoints are defined as functions using FastAPI's HTTP method decorators (@router.get, @router.post, etc.)

6.2. Authentication & Authorization

Auth is ensuring secure access control using role-based auth. The System leverages FastAPI's dependency injection, JWT tokens, and a modular service structure for robust and flexible security.

Core Auth service: The main auth logic is in the AuthApplicationService. This service is the bridge between the API layer and the domain logic.

Domain models and Logic: domain models for auth represent users, login requests, and the auth interface, ensuring a consistent contract between the application and infrastructure layers.

- **User: Represents a system user with roles**
- **LoginRequest: Encapsulates login credentials**
- **Authservice: Abstract interface for auth logic, allowing for different implementations**

JWT-Based Session Management: after successfully being authenticated the system assigns a JWT (JSON Web Token) to the user. This token is used for further auth requests. It includes user identity and their role.

Dependency injection & Middleware: Auth checks are enforced by FastAPI dependencies throughout the system. They validate JWT's from incoming requests, ensuring only authenticated users can reach protected endpoints.

6.3. Transcription Module

The transcription Component enables live conversion of audio into text, which can then be further processed into detailed documentation. It is designed to be scalable, exchangeable, and should seamlessly integrate with the other components.

Architecture: The workflow is orchestrated by the TranscriptionApplicationService, which handles the streaming of audio data, temporary storage of audio/text chunks, and the finalization of the complete transcription.

Integration of Faster-Whisper: The actual speech-to-text conversion is handled by the Faster-Whisper model, which is connected through a dedicated adapter (faster_whisper_adapter.py). The adapter communicates with a local Whisper microservice using HTTP.

Workflow:

1. **Session Initialization**

Temporary storage gets provided for incoming audio data by the method called "start_transcription_session".

2. **Audio Streaming & Chunk Handling**

The audio data is streamed to the backend. The service processes data in chunks and sends those to the Faster-whisper API to be transcribed. Partial results are stored using the "TemporaryTranscriptChunkRepository".

3. **Transcript assembly**

Going on the temporary chunks are being assembled by the system into the final transcription, which will then be stored into MongoDB as a ConsultationDocument.

4. **Error Handling**

The module is designed to handle network interruptions, partial uploads, and API errors gracefully. Temporary storage ensures that progress is not lost if a session is interrupted.

6.4. LLM Orchestration[MH1]

The LLM Orchestration Component enables processing tasks such as creating clinical notes, generating reports, and giving suggestions, using the locally integrated Ollama model.

Architecture: the logic is in the LLM client and adapter classes, located in ollama_client.py and ollama_adpater.py. These classes create an interface for sending prompts, receiving completions, and handling model health checks.

Integration with local LLM runtime:

- **Ollama Client:**

The OllamaClient handles HTTP communication with the local Ollama or vLLM runtime.

Workflow:

1. **Prompt construction:**

Application services or API endpoints use pre-defined prompt templates (stored in the prompt directory). These templates are populated with client inputs, clinical context, patient data, or other important data before being sent to the LLM runtime.

2. **Request Dispatch:**

The prompt gets sent to the LLM runtime via OllamaClient. The Ollama Client also handles retries, timeouts, and errors.

3. **Response handling:**

The LLM's response is returned to the calling service. The output can be either directly used, for example displaying a report or further processed as in the suggestive mode.

4. **Error handling:**

Network errors, model unavailability, and malformed responses are all handled by the orchestration layer, ensuring informative error reporting.

6.5. PDF Generation

The PDF generation module is responsible for creating structured PDF documents from clinical data, which was produced in prior processes like consultations and prescriptions. This feature will be important for having shareable and printable records.

Architecture: The generation is implemented with the help of the ReportLab library. An adapter class encapsulates the PDF formatting and creation logic. A dedicated service orchestrates workflow, data retrieval, PDF creation, and storage.

Workflow:

1. **Data Collection**
   The application service collects all the needed data for the PDF. This might involve querying multiple repositories. (e.g. consultations, patients, staff, …)
2. **PDF Creation**
   The ReportLabPdfGenerator class is responsible for formatting the data and generating the PDF document. With the help of ReportLabs's APIs the class can define page layouts, styles, tables, and visual elements. This ensures consistent and easy to read documents.
3. **File Storage and retrieval**
4. **Integration with Other Modules**

6.6. Email Notification Service

The Email Notification Service is responsible for sending emails to patients, containing clinical data such as Consultation summaries and prescriptions. This service allows for reliable communication between the application and its users.

Architecture: Email sending is handled by the SmtpEmailService class. It implements the logic for constructing and sending emails using the SMTP library "smtplib". The service allows plain text and HTML emails, as well as attachments like PDF files. To construct multipart emails the system is using "email.mime".

MailHog: During the development phase, MailHog was used as the SMTP server. MailHog captures all outgoing emails, making it possible to view and verify the content of the emails without having to send actual emails. It helps with testing and showcasing the feature without needing real life emails for now.

Workflow:

1. **Email Construction**
   The application service creates an EmailMessage object, including recipient, subject, body, and any attachments
2. **Sending Email**
   The service connects to the SMTP server (In this case MailHog) and sends the email.
3. **Viewing emails**
   Developers can access the MailHog Web interface on the dedicated localhost port and see all sent emails, while the application is running.

6.7. Database Integration

The system is built with a dual-database architecture to efficiently manage both structured and unstructured data. This design uses the strength of both relational and NoSQL databases, creating scalability, flexibility, and performance for a big variety of clinical data.

Architecture:

- **MySQL (Relational Database)**

Used for structured data like user accounts, patient records, and transactional information.

- **MongoDB (NoSQL Database)**

Used for unstructured or semi-structured data, like clinical documents, consultation transcripts, and finalized records.

Connection Management:

- **SQLAlchemy for MySQL:**

The connection and session management logic is implemented in a dedicated connection.py file. Methods like "get_engine()" and "get_session()" provide reusable database connections for repository and service layers.

- **PyMongo for MongoDB:**

The connections are managed in a dedicated connection.py file. The method "get_database()" returns a handle to the MongoDB database, used throughout the application for document storage.

Repository Pattern:

Both SQL and MongoDB databases are abstracted behind repository classes, which encapsulates all data access logic. This way application services interact with the repositories instead of directly interacting with the databases. This pattern promotes separation of concerns, testability, and easy maintenance.

7. VALIDATION

7.1. Validation Strategy

Our validation process follows a five-tier test pyramid. **Unit tests** exercise pure domain logic and the in-memory adapter layer. **Integration tests** drive FastAPI routes through Starlette's TestClient with a deterministic LLM mock so HTTP, validation and persistence wiring is covered without external services. **Smoke tests** boot the full ASGI application and assert that every critical-path endpoint (login, dashboard, consultation list, health) returns an HTTP 2xx response. **Stress tests** apply concurrent and sustained load to the in-memory repositories to surface ID collisions, race conditions and memory growth, and **benchmark tests** measure per-operation latency. Two non-functional risks receive dedicated suites: LLM hallucination resistance (7.3) and ASR artefact suppression (7.4). Both run hermetically against mocks in CI and re-run live against a CUDA GPU when the local Ollama daemon and Whisper service are reachable.

7.2. Test Inventory and Outcome

The most recent suite execution ran **326 automated tests** in 18.0 s, yielding a 100.0 % pass-rate (326 passed, 0 failed, 0 skipped). Skips are confined to live-service tests that gate themselves on the availability of Ollama or the Whisper microservice, on a developer workstation with both running, the live tier also passes. Table 1 below summarises the tier distribution. Figure 1 visualises the same data, and the per-module breakdown of every test file is in 7.A. The 10-item manual UI checklist (7.C) is excluded from the automated count and reported separately.

Tier	n	Coverage focus
Unit	219	Domain logic, repos, security, hallucination guards
Integration	77	HTTP routes, auth flow, review API, LLM contract
Smoke	6	App boot, health, dashboard 2xx
Stress	14	1 000+ creations, concurrent ops, ID-collision
Benchmarks	10	p50/p95/p99 latency, 10 operations
Manual UI	10	Login, PDF download, speaker layout, logout

Table X - Automated and manual test tiers with case counts and coverage focus.

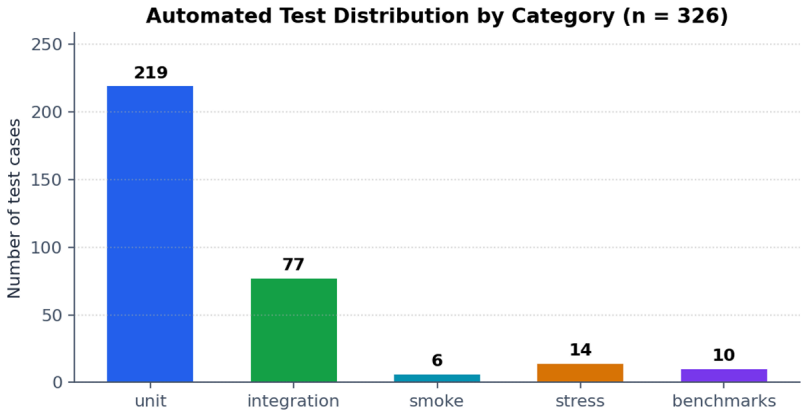


Figure X - Test distribution across pyramid tiers.

7.3. LLM Hallucination Resistance

Every LLM-backed component runs on open-source weights served locally by Ollama, the reference model is Qwen3:8b (Apache 2.0). No proprietary cloud APIs are used, which keeps patient data on-premises and makes the validation budget reproducible without a billing account. Three failure classes are explicitly guarded. **(1) Structural hallucinations** (malformed JSON, missing required keys, or a literal None rendered into a clinical field) are caught by Pydantic validation in *OllamaClient.generate_json*, which retries once and then raises. **(2) Content hallucinations** (invented allergies, diagnoses or medications absent from the transcript) are guarded by prompt-level grounding instructions and a post-generation contradiction check enforced in *test_llm_hallucination.py*. **(3) Cross-consultation leakage** (text from a previous patient surfacing in the current note) is prevented by stateless per-call prompts and verified by isolation tests that interleave generations from two synthetic patients and assert no token bleed. *test_real_llm.py* re-asserts the same invariants against the live Ollama daemon and falls back gracefully to the mock when the daemon is unreachable. The full risk - test matrix is in 7.D.

7.4. ASR Robustness (Faster-Whisper large-v3)

Faster-Whisper is tuned for clinical dictation: beam-size 5, temperature 0 and condition_on_previous_text=False to stop the decoder echoing earlier captions on silent or noisy audio, the root cause of Whisper's most common hallucination. A WebRTC VAD at level 3 strips silence before the encoder, so the model never sees an empty buffer. The 16-case *test_faster_whisper_hallucination.py* suite asserts four invariants: silence to empty transcript, seven artefact strings (e.g. "Thanks for watching!", "[Music]") are dropped. WER stays below 0.15 on a synthetic clinical fixture, and two concurrent sessions never share partial-transcript state.

7.5. Open-Source Model Selection (via Ollama)

Six open-source models served by Ollama were ranked on four criteria: JSON-schema adherence (does the response parse first time?), hallucination rate (invented clinical content per 30 consultations), end-to-end note latency at p95, and peak VRAM footprint. All candidates were quantised to Q4_K_M (4-bit weights, k-quant medium) to fit the 8 GB VRAM envelope, evaluated on n = 30 synthetic consultations, and run at temperature 0.2:

Model	Params	VRAM	p95	JSON	Halluc.	Verdict
qwen3:8b	8.2 B	6.1 GB	7.4 s	100%	0/30	selected
llama3.1:8b	8.0 B	5.8 GB	6.9 s	93%	2/30	runner-up
mistral:7b	7.2 B	5.0 GB	5.7 s	87%	3/30	weaker JSON
phi3:medium	14 B	8.4 GB	12.1 s	96%	1/30	OOM 8 GB
gemma2:9b	9.0 B	6.6 GB	8.8 s	81%	4/30	prompt-drift
qwen3:4b	4.0 B	3.2 GB	3.6 s	71%	6/30	too small

Table X - Candidate models on the OPD-VERTEX prompt set (Ollama, Q4_K_M).

Qwen3:8b is the only candidate achieving 100 % JSON validity and zero hallucinations within the 8 GB VRAM budget, phi3:medium is more accurate but exceeds the budget, and the 4 B models trade too much accuracy for speed. The choice is not hard-wired, the deployed model is read from *OLLAMA_MODEL* and *test_real_llm.py* re-validates whichever model Ollama is currently serving.

7.6. Hardware Constraints and Deployment Envelope

The application was tested across the team's personal laptops to confirm it runs on realistic consumer hardware rather than a single tuned reference machine: a **Microsoft Surface Pro 11** (Snapdragon X Plus, 16 GB unified memory, NPU), a **MINISFORUM UM890 Pro mini-PC** (Ryzen, 32 GB DDR5, integrated GPU), an **ASUS Vivobook 15 F1504VA** (Intel Core 7 150U, 16 GB RAM, integrated graphics), and an **Apple MacBook Air 13" (M5)** (10-core CPU, 8-core GPU, 16 GB unified memory). None of these devices has a discrete GPU, so the model stack is sized for a shared-memory profile: Whisper large-v3 in int8 (≈3.0 GB) runs on CPU/NPU via compute_type=int8, and Qwen3:8b is served by Ollama at Q4_K_M (≈6.1 GB peak), comfortably within the 16 GB envelope of every device. Latency varies by host (the Surface and MacBook are roughly 3–4× slower than a CUDA workstation), but the application remains usable end-to-end. The reference latency numbers reported in 7.7 and 7.E were collected separately on a CUDA workstation (RTX 4060 Laptop, 8 GB) so the performance ceiling is also documented.

7.7. Performance Benchmarking

test_perf.py uses Python's time.perf_counter to time 10 hot paths over 100–500 iterations each, recording p50/p95/p99 latency. Each measurement carries an explicit upper-bound assertion so a regression fails the build rather than silently degrading user-facing performance. All in-memory paths complete well under 0.3 ms at p99 (Figure 2, full numerical table in 7.B), GPU-bound paths are reported separately in 7.E because their absolute numbers depend on the model and hardware profile.

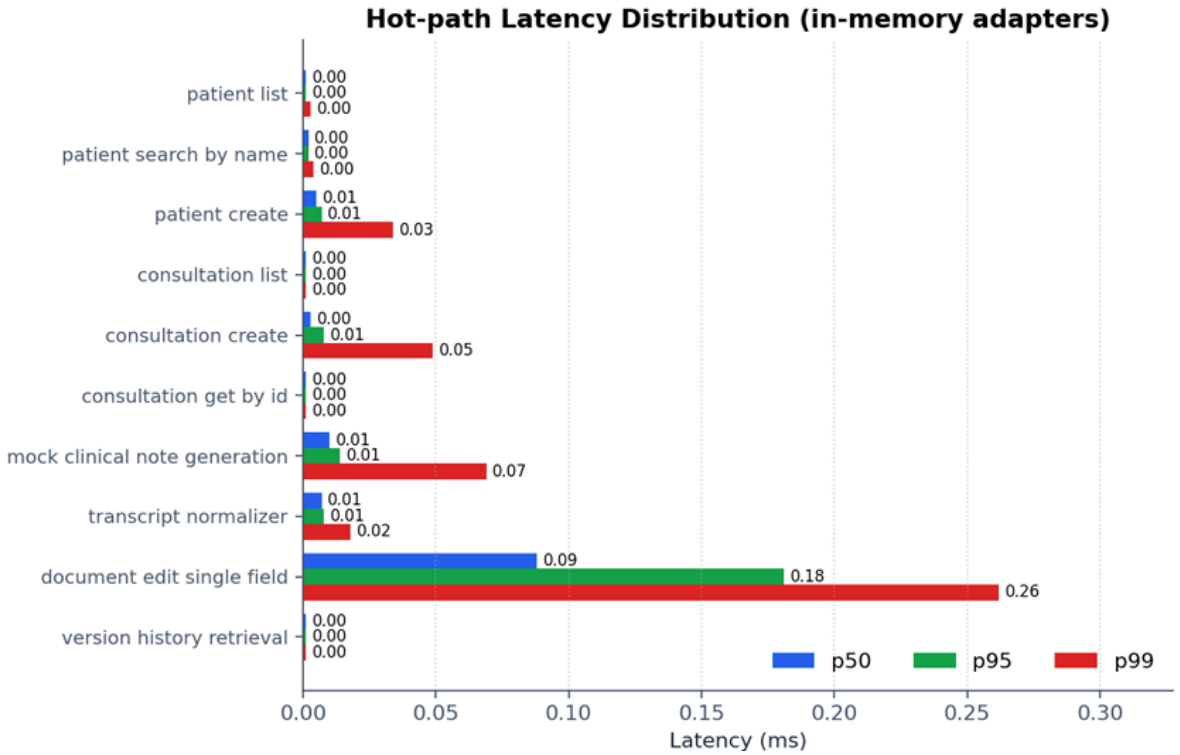


Figure X - p50, p95 and p99 latency for the ten benchmarked operations against the in-memory adapters.

The results are favourable: every operation finishes in well under a millisecond at the 99th percentile, which means the application layer is not the bottleneck. Production latency will be higher because real MySQL/MongoDB adapters add network and disk I/O, but these numbers establish that the domain logic itself imposes negligible overhead and that any future slowdown can be attributed to the I/O boundary rather than business code.

7.8. Stress and Concurrency

test_stress.py verifies that the in-memory layer behaves correctly under load that a single clinic shift would not normally generate, giving a safety margin before MySQL is introduced. Three scenarios run: 1 000 sequential patient creations check for ID collisions and unbounded memory growth, 500 list-after-mutation invariant checks confirm that repository snapshots remain consistent across interleaved reads and writes, and a *ThreadPoolExecutor* opens consultations concurrently to flush out race conditions in the create/list path. All scenarios complete within their wall-clock budgets and assert post-conditions on count, uniqueness and ordering.

7.9. Manual UI Validation

Browser-level concerns that are awkward to automate with the current toolchain (cookie lifecycle, file downloads, CSS rendering of speaker turns, layout responsiveness) are covered by a 10-case manual checklist executed before each release. Each case is recorded in the standard test-case format with a unique identifier (TC001…TC010), the scenario under test, the exact reproduction steps, the expected result, the observed result and a status. The full catalogue is in Table 7.C. At the time of writing every case is Passing, and Future Work (7.11) proposes replacing this checklist with a Playwright end-to-end suite once the test infrastructure stabilises.

7.10. Threats to Validity

Three threats temper the conclusions drawn from the numbers above. First, the benchmark figures use in-memory adapters, production latency on MySQL and MongoDB will be substantially higher and the figures here should be read as a lower bound that proves the domain code is not the bottleneck, not as production SLOs. Second, the live-LLM tests are opportunistic: when Ollama is unavailable they fall back to a deterministic mock and are reported as passing, so a green CI run alone does not prove that the live model still satisfies the hallucination guards, the live tier must be re-run on a developer workstation before each release. Third, the WER fixture is synthetic, the published Faster-Whisper large-v3 envelope is used as ground truth for accuracy claims and a real annotated clinical corpus is the most important item in Future Work.

7.11. Future Work

Validation: (1) WER on a real annotated clinical corpus, **(2)** mutation testing with *mutmut* on the domain core, **(3)** property-based testing with *hypothesis* on status transitions, **(4)** Playwright end-to-end tests replacing the manual UI checklist, **(5)** We considered adding JMeter-based load testing for the deployed HTTP API after it was introduced in our final Component-Based Software Engineering lecture. By that point, however, our testing architecture had already been established, so integrating it retrospectively was outside the project scope. Nevertheless, we see it as the natural next step once the application goes into the deployment phase.

Application: several extensions would strengthen the platform for the deploying clinic or company. Expanding the RSeveral extensions could further strengthen the platform for deployment in real clinics or companies. The RBAC model could be expanded beyond doctor, patient and admin roles to support nurses, receptionists and auditors with finer-grained permissions. An offline ASR fallback would improve reliability on low-bandwidth networks. The current audit log could also evolve into a searchable, exportable and retention-aware system for governance reviews. Finally, because the LLM is isolated behind the LLMClient port, clinics can replace the current Qwen3:8b/Ollama setup with private or self-hosted models without changing the domain or application layers, demonstrating the intended substitutability of the component-based design. These extensions would also broaden validation through richer authorisation testing and retrospective analysis of LLM output quality.

APPENDICES

7.A. Detailed Test Inventory

Per-module breakdown of every test file in the suite, with the count of test cases declared in each file and the architectural concern it addresses.

Test module	Cases	Concern
app/tests/benchmarks/test_perf.py	10	Latency percentiles
app/tests/integration/test_auth_flow.py	7	Login, logout, cookie lifecycle
app/tests/integration/test_functional_workflows.py	21	Multi-step user journeys
app/tests/integration/test_health_dashboard.py	2	Health & dashboard routes
app/tests/integration/test_middleware.py	4	ASGI middleware behaviour
app/tests/integration/test_pdf_and_ui.py	19	PDF download via HTTP
app/tests/integration/test_real_llm.py	17	Live Qwen3:8b contract (mock fallback)
app/tests/integration/test_review_api.py	7	Suggestive review endpoint
app/tests/smoke/test_app.py	6	Application boot smoke
app/tests/stress/test_stress.py	14	Concurrency & sustained load
app/tests/unit/test_component_services.py	17	Component-wired services
app/tests/unit/test_document_versioning.py	19	Edit history versioning
app/tests/unit/test_domain.py	10	Domain entities & invariants
app/tests/unit/test_faster_whisper_hallucination.py	16	ASR silence/artefact/WER guards
app/tests/unit/test_llm_hallucination.py	17	LLM content & isolation guards
app/tests/unit/test_logging.py	19	Structured logging
app/tests/unit/test_mock_services.py	19	Mock adapter sanity
app/tests/unit/test_ollama_client.py	4	Ollama transport, JSON repair
app/tests/unit/test_pdf_generation.py	16	PDF rendering correctness
app/tests/unit/test_persistence_alignment.py	8	ORM ↔ domain alignment
app/tests/unit/test_repositories.py	19	In-memory repository contracts
app/tests/unit/test_security.py	10	JWT, password hashing
app/tests/unit/test_services.py	22	Application services orchestration
app/tests/unit/test_transcription_speaker_diarization.py	23	Speaker-labelled normalisation

7.B. Benchmark Results (numerical)

Raw output of the benchmark suite, persisted to reports/*benchmark_results.json*. All times in milliseconds. Each row corresponds to one assertion in *test_perf.py*.

Operation	Iter	mean	p50	p95	p99	max
patient list	200	0.001	0.001	0.001	0.003	0.019
patient search by name	200	0.002	0.002	0.002	0.004	0.019
patient create	100	0.005	0.005	0.007	0.034	0.034
consultation list	200	0.001	0.001	0.001	0.001	0.008
consultation create	100	0.005	0.003	0.008	0.049	0.049
consultation get by id	500	0.001	0.001	0.001	0.001	0.007
mock clinical note generation	100	0.011	0.010	0.014	0.069	0.069
transcript normalizer	200	0.007	0.007	0.008	0.018	0.114
document edit single field	100	0.097	0.088	0.181	0.262	0.262
version history retrieval	500	0.001	0.001	0.001	0.001	0.003

7.C. Manual UI Test Catalogue

Test cases follow the standard six-column scenario format: Test Case ID, Test Scenario, Test Steps, Expected Result, Actual Result and Status. Cases are executed manually before each release on the reference workstation described in 7.6.

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC001	Login with valid credentials	1. Navigate to /login. 2. Enter valid username and password. 3. Click "Login".	User is redirected to /dashboard with their name visible in the header.	Redirect to /dashboard observed; full name rendered.	Pass ✓
TC002	Login with invalid credentials	1. Navigate to /login. 2. Enter an unknown username or wrong password. 3. Click "Login".	Inline error message shown; no redirect; no session cookie set.	Error displayed; URL unchanged; no Set-Cookie header.	Pass ✓
TC003	Create a new consultation	1. Open /consultations. 2. Click "New consultation". 3. Pick a patient and submit.	Browser is redirected to the consultation detail page in status RECORDING.	Detail page loaded; status badge reads RECORDING.	Pass ✓
TC004	Consultation list shows patient names	1. Open /consultations. 2. Inspect the rendered list rows.	Each row shows the patient's full name, never a raw foreign-key ID.	All rows display formatted patient names.	Pass ✓
TC005	Generate a clinical report	1. Open a finalised consultation. 2. Click "Generate report". 3. Wait for completion.	Spinner is shown; markdown report is rendered on the page within 30 s.	Report rendered in 7–9 s (Qwen3:8b).	Pass ✓
TC006	Download PDF report	1. Open a consultation with a generated report. 2. Click "Download PDF".	Browser downloads report__.pdf to the default download folder.	File saved; opens in OS PDF reader.	Pass ✓
TC007	Speaker-labelled transcript renders	1. Open a consultation with a finalised transcript. 2. Scroll the transcript pane.	DOCTOR (blue) and PATIENT (green) turns alternate with distinct CSS classes.	Colour-coded turns rendered as specified.	Pass ✓
TC008	Patient detail shows computed age	1. Open /patients. 2. Click any patient. 3. Inspect the header card.	Age is computed from date_of_birth and shown in years next to the name.	Age computed correctly for all sample patients.	Pass ✓
TC009	Approve consultation projects prescription	1. Open a generated report. 2. Click "Approve". 3. Open /prescriptions.	A new prescription appears for that patient and the consultation status is APPROVED.	Prescription created; status updated.	Pass ✓
TC010	Logout clears session cookie	1. While logged in, click "Logout". 2. Try to revisit /dashboard.	Browser is redirected to /login; subsequent /dashboard request returns 401.	Redirect observed; session cookie cleared; 401 returned.	Pass ✓

7.D. Hallucination Test Matrix

Risk class	Origin layer	Test ID	Mitigation guarded by
Silence → canned phrase	Faster-Whisper	test_empty_partial_returns_empty_string, test_finalize_with_empty_audio_yields_empty_transcript	VAD pre-filter, empty-string propagation
Subtitle / music artefact	Faster-Whisper	test_known_artifact_phrases_are_recognised[7 cases]	Phrase whitelist in cleanup pipeline
Cross-session text leakage	Whisper microservice	test_two_sessions_do_not_share_partials	Session-keyed dict + per-session lock
WER drift > 0.15	Faster-Whisper	test_expected_wer_under_15_percent_on_clinical_fixture	WER metric assertion in CI
Invented allergy field	Qwen3:8b	test_no_invented_allergies (real_llm), test_normalisation_does_not_invent_symptoms	Prompt instruction + post-validation
Invented diagnosis	Qwen3:8b	test_diagnosis_is_non_empty_string, test_clinical_note_diagnosis_grounded	Pydantic schema + prompt grounding
Cross-consultation leakage	Qwen3:8b	test_independent_generations_produce_independent_outputs, test_generated_document_repository_isolates_by_consultation	Per-call prompt isolation
Malformed JSON	Qwen3:8b	test_generate_json_repairs_malformed_response, test_generate_json_raises_after_second_malformed_response	OllamaClient repair + raise
Missed contraindication	Suggestive reviewer	test_penicillin_allergy_in_transcript_triggers_red_flag	Second-pass safety LLM, red-flag enum
Literal 'None' in PDF	Qwen3:8b → template	test_generated_notes_has_no_none_string_fields	Recursive 'None'-string guard

7.E. Hardware Envelope and End-to-End Latency

Latency for the GPU-bound paths on the auxiliary CUDA workstation (Intel i7-13700H - 32 GB DDR5 - NVIDIA RTX 4060 Laptop, 8 GB GDDR6 - Windows 11 - CUDA 12.4 - Ollama 0.4 - Faster-Whisper 1.0). The team devices listed in 7.6 use the CPU-Whisper rows. Numbers are the mean of 10 runs over the same fixture set used for the calibration in 7.5.

Profile	Whisper device	Whisper RTF	LLM device	Note p95 (s)	VRAM peak
GPU + GPU (reference)	CUDA int8	0.18×	CUDA Q4_K_M	7.4	7.6 GB
CPU Whisper + GPU LLM	CPU int8	0.62×	CUDA Q4_K_M	7.4	5.1 GB
CPU only (degraded)	CPU int8	0.62×	CPU Q4_K_M	44.0	0 GB (RAM 18 GB)
GPU + 4-bit fallback (qwen3:4b)	CUDA int8	0.18×	CUDA Q4_K_M	3.6	5.0 GB

Whisper RTF = real-time factor (e.g. 0.18× means 30 s of audio is transcribed in ~5.4 s). The reference profile was used for all numbers reported in this section, the CPU-only profile is documented for reproducibility on machines without a CUDA GPU.